

IEEE-CIS Fraud Detection

Document 2 of 3 — Feature Engineering

This document explains every feature-engineering step in **Section 3** of the notebook: what each block of code builds, why it was built that way, and what happened when it ran on the real dataset. Feature engineering takes the raw 434-column merged frame from Document 1 and turns it into the 427-column numeric matrix that actually gets fed into LightGBM, XGBoost, and CatBoost.

Starting point: train and test are concatenated into one combined frame (df, 1,097,231 rows = 590,540 train + 506,691 test) before any feature is built. **Why concatenate first:** several features below (frequency counts, group averages) need to see the full vocabulary of values across both train and test to be built consistently — otherwise a device or card seen only in test would get an undefined frequency count. Critically, **this never touches the isFraud label** — it is dropped before concatenation — so nothing here can leak the target.

3.1 Collapsing the V block (339 → 134 columns)

What the code does

The dataset ships 339 anonymized V1 . . V339 columns (Vesta's own engineered features, provided with no description). The code groups these by their **missing-value pattern** (columns with an identical NaN pattern tend to be from the same underlying feature family), then within each group computes pairwise correlation and keeps only **one representative per correlated cluster** (correlation ≥ 0.75) — specifically the column with the most distinct values, since that carries the finest-grained signal.

Why this step exists

- 339 near-duplicate columns would **slow every model down substantially** for almost no extra predictive signal — correlated columns split trees on almost the same information repeatedly.
- It also **reduces overfitting risk**: highly correlated features let a tree model latch onto whichever near-duplicate happens to have slightly less noise in a given fold, which doesn't generalize.
- The correlation check runs on a **100,000-row sample of the train portion only** — a deliberate choice so this dimensionality-reduction decision is made without looking at test data at all.

Actual output when run

kept 134/339 V columns, dropped 205 (done in 10.8s)

Roughly 60% of the V-block was redundant — over 200 columns were mathematically near-duplicates of ones already kept.

3.2 Time features

What the code does

From the raw `TransactionDT` (seconds since an arbitrary origin), the code derives: `hour` (0-23), `dow` (day-of-week proxy), `dom` (day-of-month proxy), `is_night` (a 0/1 flag for hours 0-7), and `DT_M` (a calendar-month bucket used only for the cross-validation split, dropped before training).

Why this step exists

The critical design rule here: absolute time must never become a feature. If the raw, ever-increasing `TransactionDT` value itself were fed to a model, the model would learn something like "fraud rate goes up as this counter increases" — a rule that only reflects a particular historical window and breaks the moment it's asked to extrapolate onto test, which lies entirely in the future (see Document 1, Section 3.2). *Cyclical* position — what hour of day, what day of week — is different: fraud genuinely clusters at certain hours (e.g. overnight) in a way that repeats every day/week, so it's a legitimate, non-leaking, generalizable pattern.

Actual output when run

```
month buckets: [12, 13, 14, 15, 16, 17, 19, 20, 21, 22, 23, 24]    (done in 0.7s)
```

These 12 month buckets (spanning train + test) are exactly what feeds the `GroupKFold` split explained in Document 1, Section 4.

3.3 Transaction amount features

What the code does

Builds `amt_log` (log1p of the amount, to tame its skew), `amt_cents` (the fractional/decimal part), `amt_is_round` (1 if the amount has no cents), and `amt_decimal_len` (how many decimal digits the amount has).

Why this step exists

Finding worth calling out: the fractional part of an amount is a surprisingly strong fraud signal in practice — foreign-currency conversions tend to produce long, irregular decimals, while typical human-entered or already-rounded amounts don't. This is a domain-knowledge feature, not something a raw amount column alone would expose to a tree model as easily.

Actual output when run

```
[amount features] done in 6.6s
```

3.4 Email domain features

What the code does

Raw email domains (`P_emaildomain`, `R_emaildomain`) are mapped to a normalized provider name via a hand-built lookup table (e.g. `gmail.com` and `googlemail.com` both → `google`), plus a derived suffix, a US-domain flag, and a match flag comparing purchaser vs. recipient email.

Why this step exists

- Raw domains are **high-cardinality and inconsistent** — dozens of regional Yahoo variants (`yahoo.co.jp`, `yahoo.de`, etc.) would otherwise each be treated as a completely separate category by the model, fragmenting a signal that's really "is this a Yahoo account" into dozens of near-empty categories.

- Mapping to provider **generalizes to domains only seen in test** — if a Yahoo variant appears in test that wasn't in train, the lookup table still correctly buckets it as "yahoo" rather than the model treating it as a totally unknown category.
- `email_match` (does the purchase email match the recipient email) is a classic fraud tell — mismatches are more common in account-takeover and gift-fraud patterns.

Actual output when run

```
[email features] done in 7.1s
```

3.5 Device, browser, OS, and screen resolution

What the code does

Free-text device/browser fields (`id_30`, `id_31`, `id_33`, `DeviceInfo`) are parsed with brand-keyword matching and regex splitting to separate device brand, browser name+version, OS name+version, and screen resolution is turned into a numeric screen-area proxy ($\text{width} \times \text{height}$).

Why this step exists

These fields arrive as messy free text (e.g. "SM-G960U Build/QP1A" or "chrome 78.0"). Splitting name from version turns thousands of one-off exact strings into a handful of reusable categories a tree can actually split on meaningfully, and screen area is a reasonable numeric proxy for device class (phone vs. tablet vs. desktop) that a raw resolution string wouldn't expose directly.

Actual output when run

```
[device features] done in 17.5s
```

3.6 De-trending the D-columns, and building UID

What the code does

Most `D1` . . `D15` columns represent "days since some previous event" (e.g. days since last transaction on this card). Each is de-trended by subtracting the row's own elapsed-days-since-origin value (`day_float`), producing new `D1n` . . `D15n` columns. Separately, a synthetic `uid` (and coarser `uid2`, `uid3`) is built by concatenating `card` + `address` (+ `email`) fields, as a proxy for "this is probably the same underlying customer/account".

Why this step exists

The de-trending logic, explained simply: a raw D-column like "days since account was created" keeps growing as calendar time passes — every transaction, on every account, gets a slightly bigger number just because more time has passed overall. Left raw, a tree would learn a threshold like "`D1 > 150` → less risky", a rule that is **already stale by the time the test period starts**, since by then every D-column value has shifted upward by the same 30-day gap. Subtracting the row's own elapsed-time value removes that universal upward drift, leaving only the *meaningful* part of the signal (how this specific account's timing compares to the norm) — which does generalize across the train/test time gap discussed in Document 1.

UID exists because there is no real customer-ID column in this anonymized dataset — `uid` is a best-effort synthetic stand-in built from fields that plausibly co-identify the same person/account, and it is what the group-aggregate features in Section 3.8 are computed against.

Actual output when run

```
uid cardinality: uid=350,242 uid2=368,211 uid3=17,040 (done in 14.8s)
```

3.7 Frequency encoding

What the code does

For ~15 high-cardinality categorical columns (card1-6, addr1/2, dist1/2, email fields, device fields), the code computes how often each value occurs across the combined train+test frame, and adds that count as a new numeric column — 50 count features total.

Why this step exists

A card seen 500 times behaves very differently, statistically, from a card seen exactly once — and unlike plain label-encoding (which assigns arbitrary integer IDs with no meaningful order), frequency puts rare categories numerically *near each other* in a way a tree model can actually split on meaningfully (e.g. "seen fewer than 5 times" is a real, learnable rule; "category ID < 4821" from label-encoding is not).

Actual output when run

```
50 count features (done in 16.4s)
```

3.8 Group aggregates — deviation-from-normal features

What the code does

For chosen (group column → value column) pairs — e.g. group by card1, aggregate TransactionAmt — the code computes group-level mean/std (and some nunique counts), then joins those back onto every row. 111 such aggregate features are built.

Why this step exists

The core idea, in plain terms: a \$900 charge is completely unremarkable in general, but is very unusual for a specific customer whose typical spend is \$30. A raw TransactionAmt column can't express that — but TransactionAmt / uid_mean (this transaction's amount relative to that account's own historical average) directly captures "how unusual is this for *this specific* account", which is far more discriminative for fraud than the raw amount alone. This is the single most important feature-engineering idea in the whole notebook — the model's own feature-importance ranking later (see Document 3) confirms these aggregates are the top contributor to model performance, ahead of the D-column work and frequency encoding.

Actual output when run

```
111 aggregate features -> frame (1,097,231, 453) (done in 29.2s)
```

3.9 Final cleanup — dropping dead weight, encoding what remains

What the code does

- **Always dropped:** raw identifiers and raw time columns that would not generalize (TransactionID, TransactionDT, _day, DT_M) plus the synthetic UID columns themselves (having already been used to build the group aggregates in 3.8, keeping the raw UID string as a feature would just re-introduce an overly specific identifier).

- **Near-empty / near-constant columns dropped**, judged on a 200,000-row train sample: any column >95% missing, or where a single value accounts for >98.5% of rows.
- **Everything still non-numeric** (device names, browser strings, etc.) is `pd.factorize`'d into integer codes — trees only need categories to be distinguishable from each other, not ordered, so this is sufficient (no one-hot encoding needed).
- **Infinities replaced with NaN** in any ratio-based feature that could have divided by zero (e.g. `amount ÷ group-mean` when the group mean is 0) — infinities silently break the histogram-building step inside LightGBM/XGBoost if left in.

Actual output when run

```
dropped 26 columns -> (1,097,231, 427)
factorising 37 non-numeric columns
replaced inf with NaN in 279 columns
memory 1701.4 MB -> 1589.5 MB (6.6% saved)    (done in 26.5s)
```

279 columns needed an `inf` → `NaN` fix — almost entirely a byproduct of the ratio features from Section 3.8 (dividing by a group with very few members can produce a near-zero denominator). Catching this here avoids a training-time crash deep inside a boosting library, which would otherwise be a confusing error to debug.

4. The final feature matrix

What the code does

The combined frame is split back apart at the original train/test boundary — the first 590,540 rows become `X`, the remaining 506,691 become `X_test` — and several assertions confirm the split is sound (same row counts as labels, identical columns on both sides, nothing non-numeric survived).

Actual output when run

```
X          (590,540, 427)
X_test     (506,691, 427)
features 427    memory 855 MB
```

From 434 raw merged columns (Document 1) to **427 final engineered features** after collapsing the V-block, dropping dead weight, and adding all the new derived columns above. Note this split is purely structural (train rows vs. test rows) — it is *not* the train/validation split used for model evaluation, which happens next and is explained in Document 1, Section 4 and Document 3, Section 4.

5. Summary — what feature engineering accomplished

- **V1-V339 → 134 columns:** removed redundancy without losing signal, via correlation clustering on train-only data.
- **Time features:** extracted only the cyclical, generalizable part of time (hour/day-of-week), never the raw absolute counter.
- **Amount, email, device features:** turned messy raw fields into generalizable categories and a domain-informed decimal signal.

- **D-column de-trending + UID:** removed a universal time-drift trap and built a synthetic customer proxy where none existed in the raw data.
- **Frequency encoding + group aggregates:** the two techniques doing the most real work — turning "how rare is this value" and "how unusual is this for this account" into features a tree model can split on directly. Group aggregates in particular are the single biggest driver of model performance (see Document 3's feature-importance discussion).
- **Final cleanup:** dropped dead-weight columns, encoded remaining categoricals, fixed infinities — leaving a clean 427-feature matrix, split back into X (590,540 rows) and X_test (506,691 rows), ready for the modeling phase covered in Document 3.

End of Document 2 — continue with Document 3: Modeling, Cascade & Metrics